

The Robertson Chemical Kinetics Problem

One fast process, and how it pins the whole computation

Group C

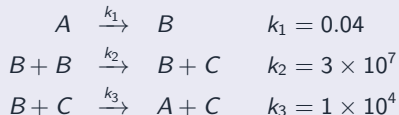
Numerical Analysis of ODEs and PDEs, Project 1, Topic 3

October 6, 2026

What this problem asks: a 1966 stiff benchmark

Problem 3: the Robertson chemical kinetics problem (classic 1966 stiff benchmark)

Three species A, B and C interconvert through three reactions:



$$\begin{aligned} y_1' &= -0.04 y_1 + 10^4 y_2 y_3 \\ y_2' &= 0.04 y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2 \\ y_3' &= 3 \times 10^7 y_2^2 \end{aligned}$$

$y(0) = (1, 0, 0)$, $t \in [0, 40]$; invariant: $y_1 + y_2 + y_3 = 1$.

The four tasks

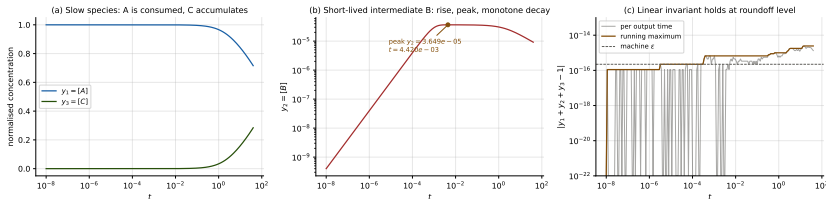
- 1 Write Python code that solves this stiff ODE system
- 2 Plot the concentration of A, B, C over time
- 3 Explain what “stiffness” means in this problem
- 4 Compare a non-stiff solver with a stiff solver

Three things have to be handed in

- the three concentration curves $y_1(t)$, $y_2(t)$, $y_3(t)$
- one specific number: $y_2(40)$
- an explanation: *why this number can be believed*

A first look at the answer: one sharp spike, two slow curves

F2 Tight-tolerance reference (Radau, tightened tolerances)



The spike in the middle is B

B jumps up from essentially zero, peaks at $t \approx 0.004$ s, then decays fast: by $t = 40$ s only 9.2×10^{-6} is left.

The two outer curves are A and C

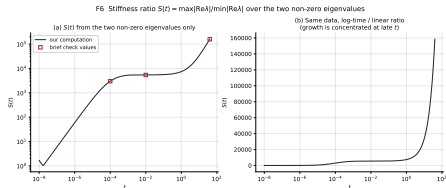
A slides from 1 down to 0.72; C rises from 0 to 0.28. Both move on a **tens-of-seconds** timescale, which has nothing to do with B.

F2: Task 2, the A, B, C concentration curves over time (reference solution, log time axis).

Remember this one picture: one spike and two slow curves – four conclusions follow, one number each.

Why it is hard (1): two worlds of different speed, this is “stiff”

Task 3



F6: the stiffness ratio $S(t)$ (fastest $\div$ slowest); the gap opens at the end.

Two sets of dots in the figure – ours and the reference values given in the assignment – land on the same curve.

The two speeds are the two eigenvalues of the Jacobian

Differentiate f with respect to y and you get the Jacobian – it lists how fast the system can move in each direction:

$$J(t, y) = \frac{\partial f}{\partial y} = \begin{pmatrix} -0.04 & 10^4 y_3 & 10^4 y_2 \\ 0.04 & -10^4 y_3 - 6 \times 10^7 y_2 & -10^4 y_2 \\ 0 & 6 \times 10^7 y_2 & 0 \end{pmatrix}$$

At $t = 40$ its two eigenvalues are one fast, one slow:

$$\lambda_{\text{fast}} = -3392.79, \quad \lambda_{\text{slow}} = -0.0214 \quad (\text{s}^{-1})$$

$$S(t) = \frac{|\lambda_{\text{fast}}|}{|\lambda_{\text{slow}}|} \rightarrow 1.58 \times 10^5 \quad (t = 40)$$

“Stiff” in plain words

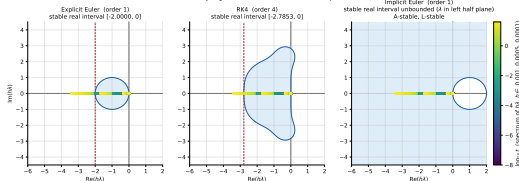
One very fast process and one very slow process standing side by side, with a large gap between them. The fast world has to be computed together with the slow one, and that is the annoying part.

Why it is hard (2): the consequence, the step size has a ceiling

Why explicit methods have a step ceiling

Explicit Euler and RK4 use only the slope at the left end.
If the step is too large, the error is **amplified** at every step,
so each method has a *safe range* of step sizes.

F4 Absolute stability regions and the actual Robertson spectrum $h\lambda(t)$



F4: safe zone from the test equation $y' = \lambda y$: one step multiplies y by $R(z)$, $z = h\lambda$; shaded = $|R(z)| \leq 1$.
Negative real axis: EE safe to $z = -2$, RK4 to $z \approx -2.79$, implicit Euler everywhere.

method	step ceiling (s)	order
Explicit Euler	5.9×10^{-4}	1
RK4	8.2×10^{-4}	4
Implicit Euler	none	1

In one line: **ceiling** = **stability edge** $\div$ **fastest eigenvalue**,
 $|\lambda| = 3392.79 \text{ s}^{-1}$ (at $t = 40$).

Steps needed to cover 40 seconds

With a ceiling of 5.9×10^{-4} s, 40 seconds divided by it gives at least 6.8×10^4 steps.

Step one: how do we know the ruler can be trusted?

Three independent ways to cross-check

- **Tighten repeatedly:** tighten the tolerance again and again, and see at which digit the number stops moving
- **A different author:** compare against two off-the-shelf SciPy solvers (BDF, Radau)
- **Structural self-check:** the problem has a conservation law of its own; use it as an extra audit

The three routes fail in completely different ways, so only if they *all* agree is the answer really trustworthy.

One number, three independent confirmations – *then* we start measuring. **First a trusted ruler, then “how accurate”.**

All three agree – that is what makes it a ruler

The three routes agree, so we take that solution as the *reference*: every “error” plotted later is a point-by-point subtraction against it.

The answer the reference gives

$$y_2(40) = 9.1855 \times 10^{-6}$$

Step two: three methods, and how each one takes a step

Explicit Euler: only looks at now

$$y_{n+1} = y_n + h f(t_n, y_n)$$

Uses *only* the slope at the left end: one f evaluation, cheap, but a large step wrecks it.

RK4: samples the step before committing

$$k_1 = f(t_n, y_n), \quad k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} k_1\right),$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} k_2\right), \quad k_4 = f(t_n + h, y_n + h k_3),$$

$$y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4).$$

Four slopes (left end, midpoint twice, right end), combined by a weighted average: four f evaluations per step, and **4th-order accuracy**.

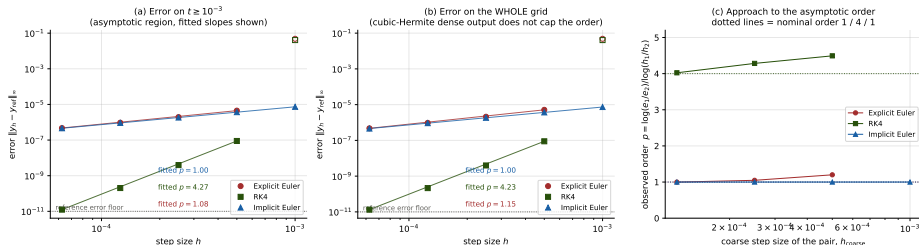
Implicit Euler: looks ahead, must solve an equation

$$y_{n+1} = y_n + h f(t_{n+1}, y_{n+1})$$

The unknown y_{n+1} sits on *both* sides, so every step solves a nonlinear system with **Newton's method**. Each step costs more, but the step itself may be *much* larger.

Step two (cont.): do our three solvers really have the right order?

F8 Uniform-step convergence, observed order and its limits



F8: error against step size, log-log; the slope of each line is the order.

Every dot is one run with a fixed h ; a p -th-order method is a straight line of slope p , so the order is not claimed, it is **measured from the fit**.

method	textbook	measured
EE	1	1.08
RK4	4	4.27
IE	1	1.00

All three match the textbook: the implementations are **correct**.

Step three: three checks plus the endpoint, one failure each

1 Conservation

What it checks: whether $y_1 + y_2 + y_3$ stays at 1

Why separately: this quantity is **identically 1** in this problem, so it cannot detect an accuracy problem at all (conclusion 2)

2 Non-negativity

What it checks: whether a concentration ever goes negative

Why separately: concentrations are physical; once one turns negative the solution is *meaningless*

3 Accuracy

What it checks: point-by-point against the reference, taking the largest error

Why separately: the only one actually measuring precision, and the only one that spots “looks fine, is wrong”

4 Endpoint

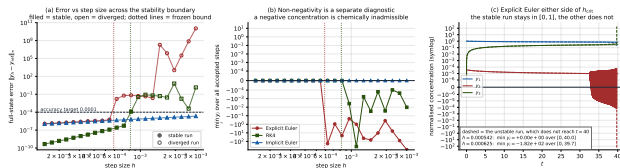
What it checks: the one number the assignment asks for, $y_2(40)$, compared directly with the reference

Why separately: it is the number on the front of the report, so it gets its own check

The first three checks are **mutually independent**, and the endpoint value is checked once more on its own. The nastiest combination is “the first two pass, the third is way off”; conclusion 2 nails it down with data.

Conclusion 1: it really is stiff, and the bound can be predicted

F7 Frozen-Jacobian estimate vs an empirical step sweep



F7 Each dot is one run (22 step sizes per method); a *filled* dot finished, an *open* dot blew up; dotted lines are the predicted limits.

units of 10^{-4} s	EE	RK4
predicted limit	5.90	8.21
largest step that survived	5.42	8.31
first step that blew up	6.25	9.58

Why the limit can be predicted

Locally the system behaves like $y' = \lambda_{\text{fast}} y$: the step is safe as long as $h|\lambda_{\text{fast}}|$ stays inside the method's stability edge (p5: EE 2, RK4 2.79). So

$$h_{\text{max}} = \frac{\text{edge}}{|\lambda_{\text{fast}}|} = \begin{cases} 2/3392.79 = 5.9 \times 10^{-4} \text{ s} & (\text{EE}) \\ 2.79/3392.79 = 8.2 \times 10^{-4} \text{ s} & (\text{RK4}) \end{cases}$$

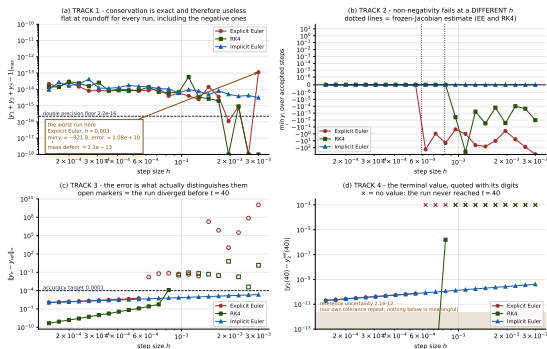
and since $|\lambda_{\text{fast}}|$ is largest at $t = 40$, this is the tightest limit for the whole run.

Only 8% off: a one-line calculation gives the step limit in advance.

Conclusion 2: surviving is not correct, and only one check bites

Task 1

F9 Four diagnostics, four verdicts: conservation cannot stand in for accuracy



F9 The same 22 runs, one check per panel; conservation sits near 10^{-13} throughout, collapsed runs included.

The most striking single run (explicit Euler, step 3×10^{-3})

conservation = 1.1×10^{-13} (looks perfect)

min concentration = -921.8 (negative!)

error = 1.08×10^{10}

Checking conservation alone would have waved this through.

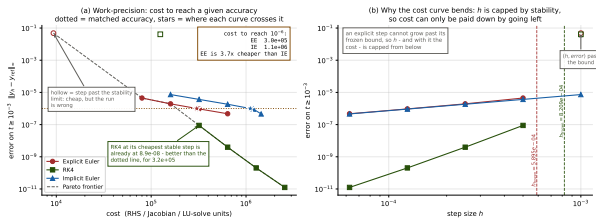
Why conservation always holds

Add the three right-hand sides: every term shows up once positive and once negative, so the terms cancel in pairs:

$$\begin{aligned} y_1' + y_2' + y_3' &= (-0.04y_1 + 0.04y_1) \\ &\quad + (10^4y_2y_3 - 10^4y_2y_3) \\ &\quad + (-3 \cdot 10^7y_2^2 + 3 \cdot 10^7y_2^2) = 0 \end{aligned}$$

So $(y_1 + y_2 + y_3)' = 0$: the sum keeps its initial value 1 no matter how accurate the run is, which is why **this check can never fail**.

F12 Cost at matched accuracy - the implicit method is not automatically cheaper



F12: cost to reach a given accuracy (work counted as one f , one Jacobian, one solve, each worth 1).

At 10^{-6} error: explicit Euler costs **3.0**, RK4 costs 3.2, implicit Euler costs **11**.

How to read it

Implicit and explicit Euler are both 1st order: better accuracy needs a smaller step, and an implicit step costs $3\times$ as much – so at high accuracy it loses. **Implicit buys a bigger step, not accuracy.**

Every step solves an equation: in $y_{n+1} = y_n + h f(y_{n+1})$ the unknown sits on *both* sides, so we rewrite the step as a root-finding problem,

$$F(y) \equiv y - y_n - h f(y) = 0$$

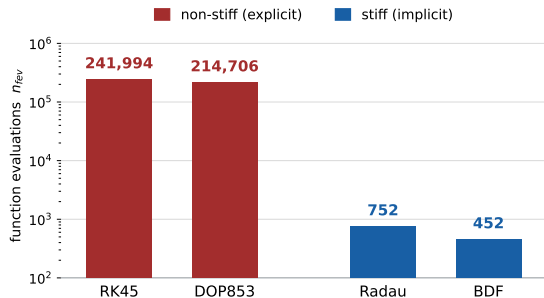
How we solve it – Newton iteration: starting from the predictor, take one linear approximation with the Jacobian $J = \partial f / \partial y$ – J is the 3×3 matrix from page 4, evaluated at $t = t_{n+1}$. The step then becomes a 3×3 linear system; the correction it gives is added to the current value and the process repeats. With only three unknowns the system is tiny, so a few iterations drive the residual down to machine precision.

A real first step: starting from the extrapolated predictor, the residual $|F|$ falls this fast in 4 iterations:

predictor	iteration 2	iteration 4
4.80×10^{-5}	1.65×10^{-7}	1.01×10^{-16}

This is why implicit costs more

Each iteration costs one f , one Jacobian, one solve, so one implicit step is about 3 times an explicit one – that is where “not cheaper” comes from.



F17 Same $y_2(40)$, same tolerances ($rtol = 10^{-6}$): how many function evaluations each solver needs, log axis.

Stiffness is the problem; explicit and implicit are the method

The two red bars, RK45 and DOP853, are non-stiff solvers and use *explicit* methods; the two blue bars, BDF and Radau, are stiff solvers and use *implicit* ones.

Why the stiff solvers are so much faster

Both get $y_2(40)$ right, but RK45 needs 2.4×10^5 evaluations against 452 for BDF: every explicit step is held below 10^{-4} s by the stability ceiling, so 40 seconds takes hundreds of thousands of steps, while an implicit method has no such limit.

Rule of thumb

Hand a stiff problem straight to an implicit solver.

Four conclusions, one number each

	conclusion	the number
1	it really is stiff	predicted 5.90, measured 5.42 (10^{-4} s), 8% apart
2	surviving is not correct	conservation 1.1×10^{-13} , yet $\min y_i = -921.8$
3	implicit is not cheaper	at 10^{-6} error: EE 3.0, IE 11
4	a stiff solver is $100\times$ faster	RK45 241,994 evaluations, BDF 452

The one number we have to hand in

$y_2(40) = 9.1855 \times 10^{-6}$. The peak 3.65×10^{-5} (at $t \approx 0.004$ s) is 4 times larger than the endpoint, so we report both.

Thank you – questions welcome